Island Chill MES

Installation & Deployment Guide

Production deployment of the Manufacturing Execution System, including ERPNext integration, database setup, reverse proxy, SSL, backup, and monitoring.

Version 2.0 | October 2026

Table of Contents

1. Architecture Overview
2. Server Requirements
3. Software Prerequisites
4. ERPNext Preparation (Before MES Deployment)
5. Database Setup (MySQL)
6. Application Deployment
7. Environment Configuration
8. ERPNext Connection Setup
9. Reverse Proxy & SSL (Nginx)
10. Process Manager (PM2 / systemd)
11. Initial Data Load & First Login
12. Backup & Recovery
13. Monitoring & Logging
14. Security Checklist
15. Upgrade Procedure
16. Troubleshooting

1. Architecture Overview

The production deployment has three tiers: the MES application (React frontend + Node.js API), a MySQL database, and the ERPNext instance (existing or new). The MES server must reach ERPNext over HTTPS.

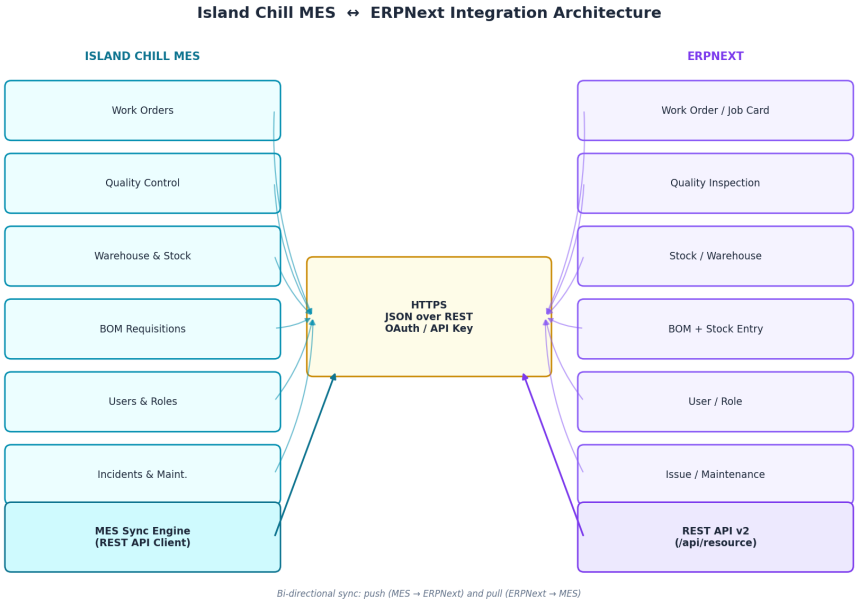


Figure 1: Deployment architecture — MES connects to ERPNext via REST API

2. Server Requirements

Component	Minimum	Recommended
MES app server	2 vCPU, 4 GB RAM, 40 GB SSD	4 vCPU, 8 GB RAM, 80 GB SSD
Database (MySQL)	Same host OK for <20 users	Dedicated host, 4 GB RAM
ERPNext	Existing ERPNext v14/v15	Frappe Cloud or 4 vCPU / 8 GB self-hosted
Network	HTTPS 443 between MES ↔ ERPNext	Site-to-site VPN if ERPNext is on-prem
Clients	Any modern browser (Chrome/Edge/Firefox)	Chrome, 1080p+ display

3. Software Prerequisites

Install the following on the MES server (Ubuntu 22.04 LTS assumed; adjust for your distro):

```
# System packages sudo apt update && sudo apt install -y nodejs npm nginx mysql-server # Node.js 20 LTS (via NodeSource) curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash - sudo apt install -y nodejs # Verify node -v # v20.x npm -v # 10.x mysql --version
```

Optional: PM2 for process management (npm install -g pm2), and Certbot for Let's Encrypt SSL (sudo apt install certbot python3-certbot-nginx).

4. ERPNext Preparation (Before MES Deployment)

Complete these steps in ERPNext first — the MES needs them to connect. Full details with screen-level steps are in the ERPNext Integration & Administration Guide (Part B).

#	ERPNext task	Reference
1	Confirm ERPNext v14/v15 is running and reachable over HTTPS	Integration Guide A.1
2	Create/verify the company record and chart of accounts	Integration Guide B.2
3	Create warehouses (stores, WIP, quarantine, finished goods, scrap)	Integration Guide B.3
4	Create UOMs and conversion factors	Integration Guide B.4
5	Create items (raw materials, packaging, finished goods) with batch/expiry	Integration Guide B.5
6	Create users and assign roles; set custom field in_mes on users that should appear in the MES	Integration Guide B.7
7	Create and submit BOMs per finished-good variant	Integration Guide B.8
8	Create workstations/operations for filling lines	Integration Guide B.10
9	Create quality inspection templates and link to items	Integration Guide B.11
10	Register equipment as Assets; create maintenance schedules	Integration Guide B.12
11	Create the integration user, generate API key/secret	Integration Guide C.1
12	Add custom_mes_reference custom field to Stock Entry, Quality Inspection, Issue, Work Order	Integration Guide Part E

5. Database Setup (MySQL)

Create a dedicated database and user for the MES. The schema is managed by Drizzle ORM migrations included in the application.

```
sudo mysql -u root CREATE DATABASE islandchill_mes CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci; CREATE USER 'mes_user'@'localhost' IDENTIFIED BY 'STRONG_PASSWORD_HERE';
GRANT ALL PRIVILEGES ON islandchill_mes.* TO 'mes_user'@'localhost'; FLUSH PRIVILEGES; EXIT;
```

Record the credentials — they go into the .env file in step 7. Run migrations after the first deploy (step 6) with npm run db:push (Drizzle Kit).

6. Application Deployment

The MES ships as a single repository with frontend (React/Vite) and backend (tRPC/Hono) together.

```
# 1. Get the code git clone https://git.yourcompany.example/islandchill/mes.git /opt/islandchill-mes cd /opt/islandchill-mes # 2. Install dependencies npm install # 3. Build the frontend (outputs to dist/) npm run build # 4. Apply database schema npm run db:push # 5. Start (dev check) — see section 10 for production process manager npm run start
```

The API listens on port 3000 by default and serves the built frontend from dist/. Nginx (section 9) proxies public traffic to it.

7. Environment Configuration

Copy .env.example to .env and fill every value. Never commit .env to source control.

Variable	Description	Example
DATABASE_URL	MySQL connection string	mysql://mes_user:PASS@localhost:3306/islandchill_mes
PORT	API listen port	3000
NODE_ENV	Runtime mode	production
JWT_SECRET	Session signing secret (generate: openssl rand -hex 32)	••••••••••••••••
ERPNext_URL	ERPNext base URL	https://erp.islandchill.example
ERPNext_API_KEY	Integration user API key	1a2b3c4d...
ERPNext_API_SECRET	Integration user API secret	••••••••••
ERPNext_COMPANY	Company name in ERPNext	Island Chill Beverages Ltd
SYNC_MODE	realtime | scheduled | manual	realtime
SYNC_PULL_INTERVAL_MIN	Master-data pull interval	60
MAIL_HOST / MAIL_FROM	SMTP for alerts (sync failures, dead letters)	smtp.example.com

Restart the application after any .env change: pm2 restart mes or sudo systemctl restart mes.

8. ERPNext Connection Setup

Once the application is running and .env is filled, verify the ERPNext connection from the MES admin UI:

Step	Action
1	Log into the MES as an Administrator.
2	Open Admin → Settings → Integrations → ERPNext.
3	Confirm URL, API key, and API secret are loaded from the environment (masked). Click Test Connection.
4	On success the page shows the connected company and user. If it fails, the banner shows the exact ERPNext error — fix per the Integration Guide Part F.
5	Click Run Full Master-Data Pull — warehouses, items, UOMs, BOMs, and users sync into the MES. This can take a few minutes on first run.
6	Verify: open Warehouse (zones populated), Admin → Products (variants from ERPNext items), Admin → Users (synced accounts with ERPNext badge).
7	Set sync mode and pull interval. For production: realtime for stock movements, 60 min for master data.

9. Reverse Proxy & SSL (Nginx)

Create /etc/nginx/sites-available/mes with the following (replace your domain):

```
server { listen 80; server_name mes.islandchill.example; location / { proxy_pass http://127.0.0.1:3000; proxy_http_version 1.1; proxy_set_header Upgrade $http_upgrade; proxy_set_header Connection 'upgrade'; proxy_set_header Host $host; proxy_set_header X-Real-IP $remote_addr; proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for; proxy_set_header X-Forwarded-Proto $scheme; proxy cache bypass $http upgrade; } }
```

```
sudo ln -s /etc/nginx/sites-available/mes /etc/nginx/sites-enabled/ sudo nginx -t && sudo systemctl reload nginx # SSL certificate sudo certbot --nginx -d mes.islandchill.example
```

Certbot rewrites the site to listen on 443 with a redirect from HTTP. Auto-renewal is installed by default; verify with sudo certbot renew --dry-run.

10. Process Manager (PM2 / systemd)

Option A — PM2 (recommended for Node.js):

```
npm install -g pm2 cd /opt/islandchill-mes pm2 start npm --name mes -- run start pm2 save pm2 startup # follow the printed command to auto-start on boot
```

Option B — systemd unit (/etc/systemd/system/mes.service):

```
[Unit] Description=Island Chill MES After=network.target mysql.service [Service] Type=simple User=mes WorkingDirectory=/opt/islandchill-mes ExecStart=/usr/bin/npm run start Restart=always RestartSec=5 Environment=NODE ENV=production [Install] WantedBy=multi-user.target
sudo systemctl daemon-reload && sudo systemctl enable --now mes
```

11. Initial Data Load & First Login

After the master-data pull (step 8.5), the MES contains real ERPNext data. Sign in with the admin account created during deployment, then: verify the dashboard loads, open a BOM Requisition, run a test QC check, and confirm each appears in ERPNext (Stock Entry / Quality Inspection). This end-to-end smoke test validates the entire integration before go-live.

12. Backup & Recovery

What	How often	Command / method
MySQL database	Daily (nightly cron)	mysqldump -u mes_user -p islandchill_mes > backup_$(date +%F).sql
.env file	On change	Copy to encrypted vault (contains secrets)
Application code	On deploy	Git tag each release; keep last 10 releases on server
Uploaded files (photos, evidence)	Daily rsync	rsync -av /opt/islandchill-mes/uploads/ backup:/backups/mes/uploads/
ERPNext	Per ERPNext policy	ERPNext's own backup (Frappe Cloud does this automatically)

Restore test: monthly, restore the latest dump into a scratch database and verify the MES starts against it in staging.

13. Monitoring & Logging

Area	Tool / location
Process uptime	pm2 monit, or systemctl status mes
Application logs	pm2 logs mes, or journalctl -u mes -f
Sync health	MES Admin → Settings → Integrations → Sync Log & Integration Health report
Nginx access/error	/var/log/nginx/access.log and error.log
SSL expiry	certbot timer (systemctl list-timers | grep certbot)
Disk / memory	htop, df -h; alert at 80% disk

14. Security Checklist

Item	Status target
HTTPS everywhere (MES and ERPNext)	✔ enforced, HTTP→HTTPS redirect
Secrets only in .env / vault, never in git	✔ verified with git log -p -- .env (should be empty)
ERPNext API keys on dedicated integration user	✔ not a human's daily account
MySQL user restricted to MES database	✔ no root app access
Firewall: 22/443 open; 3000 and 3306 blocked externally	✔ ufw status verified
Automatic security updates	✔ unattended-upgrades installed
MES admin accounts use strong passwords + optional TOTP	✔ enforced in Admin → Settings
Regular off-site backups, restore-tested	✔ monthly

15. Upgrade Procedure

1. Announce a maintenance window (typically <5 minutes).
2. Backup: mysqldump + tar the current release directory.
3. git fetch && git checkout vX.Y.Z (or pull main).
4. npm install && npm run build && npm run db:push (migrations are additive/backward-compatible).
5. pm2 restart mes (or systemctl restart mes).
6. Smoke test: login, dashboard, one stock movement sync.
7. Rollback if needed: restore the previous directory + database dump, restart.

16. Troubleshooting

Symptom	Fix
npm run build fails	Confirm Node 20+ (node -v); delete node_modules and package-lock.json, npm install again.
db:push fails	Check DATABASE_URL; confirm MySQL is running (systemctl status mysql) and credentials work (mysql -u mes_user -p).
App starts but page is blank	Check API logs; confirm dist/ exists and was rebuilt; verify nginx proxies to the right port.
502 Bad Gateway from nginx	MES process is down — pm2 status / journalctl -u mes; restart it.
ERPNext Test Connection fails	See Integration Guide Part F (401/403/timeout causes and fixes).
Sync queue growing	Admin → Settings → Integrations → Sync Log for the failing record; fix cause, then requeue.
SSL certificate fails to renew	sudo certbot renew --dry-run; check DNS points to this server and port 80 is open.

Deployment Complete

With the MES deployed, connected to ERPNext, backed up, and monitored, the system is ready for production use. Keep the two companion documents at hand: the Functions & Processes guide for module reference, and the ERPNext Integration & Administration Guide for all ERPNext-side configuration and sync behavior.